🧪 Unit Testing in Angular with Jasmine & Karma

Beginner ➜ Pro Guide

“Unit testing is like checking every LEGO brick before building the Millennium Falcon. If every brick is solid, the whole spaceship won’t collapse when someone bumps the table.”

Angular ships with **Jasmine (testing framework)** and **Karma (test runner)** out of the box.
You write tests using Jasmine, and Karma runs them in a browser.

This guide walks step-by-step from **your first test** → **real production-grade testing**.

🟢 Step 1 — Preparing the Angular Project

Angular CLI already configures testing.

```
ng new unit-testing-demo
cd unit-testing-demo
ng test
```

A browser opens and runs tests like:

AppComponent should create the app ✔

What happens internally

Jasmine → writes tests
Karma → runs tests
Chrome → executes them

Your environment is ready.

🟢 Step 2 — Your First Jasmine Test

A simple TypeScript test.

```
describe('Math utilities', () => {

  it('should add two numbers', () => {
    const result = 2 + 3;

    expect(result).toBe(5);
  });

});
```

Jasmine Concepts

Function	Purpose
`describe()`	Group related tests
`it()`	Individual test
`expect()`	Assertion

Real-world analogy

Checking your **coffee machine heats water** before attempting to brew a cappuccino.

🟡 Step 3 — Testing an Angular Component

Component

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-hello',
  template: `<h1>{{ message }}</h1>`
})
```

```
export class HelloComponent {

  message = 'Hello Angular!';

}
```

Test

```
import { ComponentFixture, TestBed } from '@angular/core/testing';
import { HelloComponent } from './hello.component';

describe('HelloComponent', () => {

  let component: HelloComponent;
  let fixture: ComponentFixture<HelloComponent>;

  beforeEach(() => {

    TestBed.configureTestingModule({
      declarations: [HelloComponent]
    });

    fixture = TestBed.createComponent(HelloComponent);
    component = fixture.componentInstance;

  });

  it('should create component', () => {
    expect(component).toBeTruthy();
  });

  it('should have correct message', () => {
    expect(component.message).toBe('Hello Angular!');
  });

});
```

What TestBed does

```
TestBed
  ↓
Creates Angular testing module
```

↓
Creates component instance
↓
Allows DOM inspection

🟡 Step 4 — Testing the HTML Template

```
it('should render message in h1', () => {

  fixture.detectChanges();

  const compiled = fixture.nativeElement as HTMLElement;

  expect(compiled.querySelector('h1')?.textContent)
      .toContain('Hello Angular!');

});
```

Why `detectChanges()`?

Angular updates templates **only after change detection**.

Analogy:

Microwave
↓
Door closed
↓
Press start
↓
Heat begins

🟡 Step 5 — Testing Component Methods

Component

```
export class CounterComponent {
```

```
  value = 0;

  increment() {
    this.value++;
  }

  decrement() {
    this.value--;
  }

}
```

Test

```
it('should increase value', () => {

  component.increment();

  expect(component.value).toBe(1);

});

it('should decrease value', () => {

  component.decrement();

  expect(component.value).toBe(-1);

});
```

Analogy:

Press thermostat "+"
Temperature increases

🔵 Step 6 — Testing Angular Services

Service

```
export class UserService {

  getUser() {
    return { id: 1, name: 'Alice' };
  }

}
```

Test

```
import { TestBed } from '@angular/core/testing';
import { UserService } from './user.service';

describe('UserService', () => {

  let service: UserService;

  beforeEach(() => {

    TestBed.configureTestingModule({});

    service = TestBed.inject(UserService);

  });

  it('should return user data', () => {

    const user = service.getUser();

    expect(user.name).toBe('Alice');

  });

});
```

Real-world analogy:

Checking your **hotel booking system returns the correct reservation**.

🟣 Step 7 — Mocking Services

Components often depend on services.

Instead of using the real service, create a **mock**.

Component

```
export class DashboardComponent {

  username = '';

  constructor(private userService: UserService) {}

  ngOnInit() {
    this.username = this.userService.getUser().name;
  }

}
```

Mock Service

```
const mockUserService = {

  getUser: () => ({
    id: 99,
    name: 'Mockingbird'
  })

};
```

Test

```
beforeEach(() => {

  TestBed.configureTestingModule({

    declarations: [DashboardComponent],

    providers: [
      { provide: UserService, useValue: mockUserService }
    ]
```

```
  });

  fixture = TestBed.createComponent(DashboardComponent);
  component = fixture.componentInstance;

});

it('should set username from mock service', () => {

  fixture.detectChanges();

  expect(component.username).toBe('Mockingbird');

});
```

Analogy:

Mocking = **practice interview with a friend pretending to be the boss**.

🔴 Step 8 — Testing HTTP Requests

Use Angular testing utilities.

Service

```
getPosts() {
  return this.http.get<Post[]>('/api/posts');
}
```

Test

```
import {
HttpClientTestingModule,
HttpTestingController
} from '@angular/common/http/testing';

let httpMock: HttpTestingController;
```

```
beforeEach(() => {

  TestBed.configureTestingModule({
    imports: [HttpClientTestingModule],
    providers: [PostService]
  });

  service = TestBed.inject(PostService);
  httpMock = TestBed.inject(HttpTestingController);

});
```

Test HTTP Response

```
it('should fetch posts', () => {

  const dummyPosts = [
    { id: 1, title: 'Hello' }
  ];

  service.getPosts().subscribe(posts => {

    expect(posts.length).toBe(1);

  });

  const req = httpMock.expectOne('/api/posts');

  req.flush(dummyPosts);

  httpMock.verify();

});
```

Analogy:

Instead of real grocery trip
You simulate shopping at home

⚡ Step 9 — Testing Async Code

Component

```
delayedMessage() {

  return new Promise(resolve => {

    setTimeout(() => resolve('Done'), 1000);

  });

}
```

Test

```
import { fakeAsync, tick } from '@angular/core/testing';

it('should return delayed message', fakeAsync(() => {

  let result = '';

  component.delayedMessage().then(v => result = v);

  tick(1000);

  expect(result).toBe('Done');

}));
```

This **simulates time passing instantly**.

🧩 Step 10 — Testing Pipes

Pipe

```
transform(value: string): string {
```

```
  return value.toUpperCase();

}
```

Test

```
it('should transform text to uppercase', () => {

  const pipe = new UppercasePipe();

  expect(pipe.transform('hello'))
      .toBe('HELLO');

});
```

🧩 Step 11 — Testing Directives

Directive

```
@Directive({
  selector: '[appRedBorder]'
})
export class RedBorderDirective {

  constructor(private el: ElementRef) {

    el.nativeElement.style.border = '2px solid red';

  }

}
```

Test

```
it('should set red border', () => {

  const directive =
    new RedBorderDirective(
      new ElementRef(document.createElement('div'))
```

```
    );

  expect(directive.el.nativeElement.style.border)
      .toBe('2px solid red');

});
```

📊 Step 12 — Code Coverage

Run:

ng test --code-coverage

Coverage reports appear under:

/coverage

Target:

80%+

But remember:

Coverage ≠ Quality.

🏆 Step 13 — Professional Testing Principles

Lessons from decades in industry.

✔ Test Behavior, Not Implementation

Bad:

Testing private methods

Good:

Testing outcomes

✔ Keep Tests Small

One scenario per test

✔ Mock Dependencies

Test component
Not service implementation

✔ Avoid Fragile DOM Tests

DOM structure changes often.

Test **behavior instead**.

🧠 Testing Pyramid

E2E Tests
(Few, slow)

Integration Tests
(Some)

Unit Tests
(Many, fast)

Goal:

70% Unit
20% Integration

10% E2E

💡 Final Wisdom:

Junior developers write code that works.

Senior engineers write code that **keeps working after thousands of changes**.

Unit tests provide that guarantee.

When your test suite is solid:

Deployments become boring.
Boring deployments are successful deployments.